Tourist Safety Monitoring System

Real-time GPS tracking with irregular polygon geo-fencing, running entirely on an Android device.

Platform: Android only (Kotlin + Jetpack Compose, Material 3). **Min SDK:** 26 (Android 8.0) · **Target/Compile SDK:** 35. **Backend:** optional. The handset works standalone — geo-fence evaluation, storage and alerts all happen on the phone, with no server and no cloud needed (map *tiles* need the network; the geo-fencing maths never does). A self-contained **authority backend** (`backend/`, Flask + SQLite) and an operator **web dashboard** ship in the same repository for when an authority wants to receive events and work an SOS queue. Without a configured address the app simply keeps events in its durable outbox and says so.

> **Project folder:** `D:\TouristSafetyMonitoring-src`. The repository was renamed from `D:\TouristSafetyMonitoring` at some point; every path below uses the current name. Nothing was deleted.

---

1. What the app does

Screen	Purpose
Welcome (role chooser)	The first screen. Two plain choices: **I'm a tourist** and **I'm an authority officer**. A returning tourist with a valid session is offered one-tap *Continue* above the choices.
Tourist sign-in / Create account	One screen for both. **Sign in** with a Tourist ID and a **PIN or password** (with a show/hide control), or **create an account** (name, phone, a PIN you choose and confirm, and optional emergency contacts). The service allocates your Tourist ID, e.g. `TR-2026-00124`, shown on the success screen with a **copy** button and sign-in instructions. Every missing or wrong field is named in words; no button is ever silently greyed out.
Authority officer console	Officer sign-in (id + password, verified server-side) and then the **register of accounts**: Tourist ID, name, registration date and source, tracking status, last known position with its timestamp, open SOS count, contacts on file and the latest risk band. Officer logins are refused for tourist tokens by the *server* (403).
Home / Safety Dashboard	The single safety indicator (green / amber / red), live counters, the newest crossings, a real **GPS status** card (waiting / acquiring / live / poor accuracy / permission denied / unavailable, with fix age and accuracy), a **your account** card (Tourist ID, name, alerts waiting to send, links to contacts / settings / sign out), and an in-app banner for the latest event.

Screen	Purpose
Live GPS Tracking	The map with every saved zone, the current fix and its accuracy circle, Start/Stop, the newest fixes, a **distance-to-boundary pill**, and three live cards: **movement & fall watch** (activity + confidence, peak impact, impact score), the **approach briefing** shown when nearing a restricted area, and the **future risk prediction** (§9.6 — the horizon, the LOW/MODERATE/HIGH band, the **calibrated** probability, the contributing factors with their non-causal caveat, the model version and the prediction time, or an honest "insufficient data"/"prediction unavailable" state).
Create Geo-Fence	Draw an irregular polygon by tapping the map, with undo / clear / close-ring and validation.
Manage Geo-Fences	List, filter, enable/disable, edit and delete saved zones.
Alert History	Append-only record of every boundary crossing, filterable by zone type, event type and source.
Settings & Permissions	Runtime-permission status, tracking interval, accuracy priority, background tracking, notification toggle, the geo-fence tolerances, and **authority position reporting** (on/off + interval).
Simulation Mode	Development-only: walks a synthetic route through a test polygon. Clearly labelled and completely isolated from real GPS.
Simulated Smartwatch	A key-less software watch (SAFE / WARNING / SOS) driven by the same real GPS pipeline, with zone-entry haptics, an SOS button, and honest offline delivery status. Reached from the watch icon in the top app bar.
Authority backend (advanced)	An operator/developer screen: the service address, a real **Test connection** that actually calls `/api/v1/health` (so *Connected* is never claimed just because a field was saved), the durable delivery queue with an *upload now* button, the current tourist session and a PIN change, and exactly what the authority holds for this phone (delivery totals, latest SOS and whether it was acknowledged, the position on file). Reached from **Settings → Backend connection (advanced)**.
Emergency Contacts	The list of people to contact in an emergency, stored durably and uploaded to the authority through the same offline queue. Reached from Home or Settings.

Bottom navigation holds the five everyday destinations (Home, Tracking, Create, Zones, Alerts). The app opens on the **Welcome** screen and asks who is holding the phone; the tourist path leads to sign-in / account creation and then the safety home, the officer path leads to officer sign-in and the register of accounts. Neither path shows the other's screens, and the officer routes are refused to a tourist token by the server. Settings and the Simulated Smartwatch are reached from the top app bar; the authority-address screen is deliberately **not** in the tourist's normal path — it lives in Settings

→ *Backend connection (advanced)*. Emergency Contacts is reached from Home or Settings; Simulation is reached from Settings and only unlocks once simulation mode is switched on.

---

2. Geo-fencing: how "am I inside the polygon?" is decided

This is the core of the app, so it is deliberately the most heavily tested part.

2.1 Point-in-polygon

`PolygonGeometry.containsRayCasting` implements the classic **even-odd ray-casting** algorithm:

- a ray is cast from the point in the +longitude direction;
- each polygon edge that straddles the point's latitude contributes a crossing;
- an odd number of crossings means *inside*.

It works for **concave** polygons (an L-shaped exclusion area reports its notch as *outside*) and for either winding order. It is **never** replaced by a circular radius: a radius cannot represent an irregular boundary, and doing so would silently mis-classify every non-circular zone.

2.2 Accuracy-aware containment

`RayCastingContainmentService` wraps the raw test with a conservative policy, in this order:

1. **Fix too coarse.** If the reported accuracy radius exceeds `maxUsableAccuracyMeters` (default 100 m), the result is `UNCERTAIN`. No decision is made.
2. **On the edge.** If the point is within `boundaryToleranceMeters` of any edge, the result is `BOUNDARY`.
3. **Circle straddles the edge.** If the accuracy circle is wider than the distance to the nearest edge, the true position could be on either side, so the result is `UNCERTAIN`.
4. Otherwise ray casting runs and returns a decided `INSIDE` / `OUTSIDE`.

The accuracy circle drawn on the map is **information only** — it never determines containment.

2.3 Stateful engine and hysteresis

`GeoFenceEngine` is a plain state machine (no Android, no SQLite) that turns a stream of readings into crossings:

- **Transitions only.** An alert is raised when membership *changes*, so an hour inside a restricted area produces exactly one ENTRY, not one per fix.
- **Sticky state.** `BOUNDARY` and `UNCERTAIN` are shown to the user but never flip membership. A single bad fix cannot fabricate an alert.

- **Hysteresis.** Leaving needs a wider margin (`exitToleranceMeters`, default 15 m) than entering (`entryToleranceMeters`, default 8 m), so standing exactly on a boundary cannot produce an endless ENTRY/EXIT ping-pong.
- **Independent zones.** State is tracked per zone id, so overlapping and nested polygons are evaluated completely separately.
- **Unknown means outside.** Launching the app while already inside a zone correctly raises an ENTRY.

2.4 Zone types

Type	On entry	Status colour
`RESTRICTED`	Recorded + warning notification	Red
`DANGEROUS`	Recorded + high-priority alert	Red (highest priority)
`SAFE`	Recorded only, informational	Green

Exits are always informational. **Safe zones never trigger the emergency path**, and an ordinary geo-fence crossing never triggers an SOS.

---

3. Data model (Room / SQLite)

Four tables, all local:

- `geo_fences` — name, description, type, ordered vertex ring (encoded as `lat,lon;lat,lon;…`), vertex count, enabled flag, created/updated timestamps.
- `location_records` — every fix with latitude, longitude, accuracy, timestamp and **source** (`GPS` or `SIMULATION`).
- `geo_fence_alerts` — every crossing, carrying its own copy of the zone name and type so history stays accurate even if the zone is later renamed or deleted.
- `outbox_events` — the durable offline queue (zone crossings and SOS) awaiting delivery. See §3.2.

`source` is stored on every location and alert row. That single column is what makes it structurally impossible for a simulated coordinate to be presented as — or to overwrite — a real one.

`PolygonCodec` is a dependency-free `lat,lon;lat,lon` encoder. It is ordered (position in the string is the winding order), lossless (`Double.toString` round-trips exactly and is locale-independent) and fail-soft (a corrupt row yields the readable vertices instead of throwing).

3.1 Migration strategy

- `AppDatabase` is at **version 3** and `exportSchema = true`; the schema is checked in under `app/schemas` (`1.json`, `2.json` and `3.json`).
- **There is deliberately no `fallbackToDestructiveMigration()`.** A destructive fallback would silently delete every saved zone and the whole alert history on a schema change — exactly the failure the brief forbids.
- `DatabaseMigrations.ALL` is wired into the builder from day one and holds two real **additive** migrations: `MIGRATION_1_2` creates the `outbox_events` table and `MIGRATION_2_3` creates the `emergency_contacts` table. No existing column changes, so every saved zone, GPS row, alert and queued event survives the upgrade. The JVM suite **executes** both migrations on a real SQLite database and asserts a pre-upgrade row survives.
- Forgetting a migration therefore fails loudly in development instead of quietly wiping a tourist's zones in the field.

3.2 Offline outbox (`outbox_events`)

Nothing leaves the device directly: every zone crossing and every SOS is first written to the `outbox_events` table, so "store now, deliver later" survives a tunnel with no signal.

- `dedupe_key` carries a **unique** index, so `INSERT OR IGNORE` makes enqueueing the same crossing exactly once — this is what prevents duplicate events during a retry storm.
- A row is `PENDING`, `SENT` or `FAILED`. A transient failure stays `PENDING` and increments `attempt_count`; a permanent rejection becomes `FAILED`. `origin` records whether the event came from real GPS, the simulator, or the **simulated watch** (`SIMULATED_WATCH`).
- `OutboxSyncer.flush()` drains the queue through a `SyncTransport` interface. This build ships a `SelectingSyncTransport` that uses the real `AuthoritySyncTransport` when an authority address is configured (see §7.2 and `backend/README.md`) and the honest `NoBackendTransport` otherwise, so events stay visibly queued rather than pretending to be delivered when no server is configured. A permanently rejected event is marked `FAILED` with the server's reason; anything that could be a misconfiguration (401, 5xx, no network) stays queued.
- A background retry loop re-attempts the queue every 30 s (a production build would also hook `ConnectivityManager`).

---

4. Permissions and the background service

Permission	When it is asked for
`ACCESS_FINE_LOCATION` / `ACCESS_COARSE_LOCATION`	First time tracking is started, and from Settings.
`ACCESS_BACKGROUND_LOCATION`	Only when *background tracking* is switched on.
`POST_NOTIFICATIONS` (API 33+)	Alongside the location request.

Permission	When it is asked for
`FOREGROUND_SERVICE`, `FOREGROUND_SERVICE_LOCATION`	Manifest-declared; required by the service.

`LocationTrackingService` is a `LifecycleService` with `android:foregroundServiceType="location"`. It calls `startForeground()` in `onCreate()` (so Android never raises `ForegroundServiceDidNotStartInTimeException`), mirrors the tracking pipeline into a persistent low-importance notification, and shuts itself down the moment tracking stops.

4.1 Truthful background-service status

The dashboard no longer guesses. `TrackingSnapshot.backgroundTracking` is only the *intent* recorded at start; the real state comes from `TrackingController.foregroundServiceRunning`, which the service flips to `true` only once `startForeground()` has actually succeeded and back to `false` in `onDestroy()`. The Home screen therefore shows one of three honest values:

Dashboard value	Meaning
`Running`	The persistent notification is genuinely up.
`Paused — needs background permission`	Tracking is on and background tracking is enabled, but the service is not running (in practice: "*Allow all the time*" was never granted).
`Off`	Background tracking is not in play.

So "Tracking: Active / Background service: Paused — needs background permission" is now the correct, explained state instead of a contradictory "Tracking Active / Background service Not running".

Android version rules, encoded in `LocationPermissions`:

- API 26–28 — background location does not exist as a separate grant; foreground is enough.
- API 29 — background location can be requested through a normal dialog.
- API 30+ — the platform refuses to show a background-location dialog at all. The app sends the user to its own Settings screen and asks for "*Allow all the time*".
- API 33+ — `POST_NOTIFICATIONS` is a runtime permission.

Battery and OEM realities. Geo-fence *decisions* stay correct regardless of power management because they are made from whatever fix arrives; only timing degrades. However Doze/App Standby and aggressive OEM task-killers (Xiaomi/MIUI, Huawei/EMUI, OnePlus/OxygenOS, Samsung One UI, Oppo/Vivo) can reduce fix frequency or stop a backgrounded app. During testing, exempt the app from battery optimisation and, on MIUI, also enable Autostart. That cannot be done from inside the app.

---

5. Simulation mode

Simulation exists so the alert pipeline can be demonstrated without walking outside. It is quarantined by design:

- it owns its **own** `GeoFenceEngine`, so simulated crossings cannot alter the membership state the GPS pipeline is tracking;
- every reading it produces is stamped `LocationSource.SIMULATION`, on the database row and on any resulting alert;
- it never touches the Fused Location Provider;
- the route is generated by `SimulationPathBuilder`, which **scans** for a genuinely interior point (an average of vertices would land in the notch of a concave polygon) and produces a path that is provably *outside → inside → outside*;
- "Delete simulated data" removes **only** rows whose source is `SIMULATION` — real GPS history is filtered out by that column and is never at risk.

Every simulation screen carries an unmissable `SIMULATION — NOT REAL GPS` badge.

5.1 Simulated smartwatch

There is **no physical wearable**. `WatchScreen` is a software watch that consumes the *same* real `TrackingSnapshot` as the dashboard (there is no second, faked data source) and shows **SAFE / WARNING / SOS**:

- entering a restricted or dangerous zone requests the **WARNING** haptic — a short double buzz — exactly once per entry, never on every fix;
- the SOS button requests a distinct, long **SOS** vibration pattern and writes a durable SOS event to the outbox;
- the SOS delivery indicator is derived from the real outbox and the server's verdict, so with no authority address configured it reads **"Queued (offline)"** rather than pretending to have been delivered, and with one configured it shows the real outcome;
- the face is permanently badged `SIMULATED WATCH — NOT PHYSICAL HARDWARE`, and the event's `origin` is stamped `SIMULATED_WATCH`.

Haptics go through a `WatchHaptics` interface: `AndroidWatchHaptics` uses `VibratorManager` / `Vibrator` (`VIBRATE` is a normal, install-time permission), while tests substitute a recording fake. An **emulator has no vibrator**, so the vibration patterns can only be felt on a physical phone — this is documented rather than hidden.

---

6. Configuring the Google Maps API key

The key is never committed. Resolution order, first non-blank wins:

1. Gradle property — `-PMAPS_API_KEY=…` or `MAPS_API_KEY=…` in `gradle.properties`
2. Environment variable — `MAPS_API_KEY`
3. `local.properties` — `MAPS_API_KEY=…` (git-ignored)

To set it up:

1. In the Google Cloud Console, create (or pick) a project.
2. **Enable** "Maps SDK for Android".
3. Create an API key under *Credentials*. Restrict it to **Android apps** and add your signing certificate's SHA-1 plus the package name `com.touristsafety.app`.
4. Put it in `local.properties`:

```
properties sdk.dir=D\:\\AndroidTools\\Sdk MAPS_API_KEY=AIza...your-key...
```

The app builds and runs without a key. When none is configured, `BuildConfig.MAPS_API_KEY` is empty and the map surfaces render a fully working **key-less OpenStreetMap map** instead of a "Map not available" placeholder — see §6.1. Tracking, geo-fencing, persistence and alerts always work, regardless of the map, because containment maths never touches a map SDK.

6.1 Key-less OpenStreetMap fallback

So the Create Geo-Fence screen is never dead just because no Google key was configured, the app ships a self-contained fallback map:

- `OfflineOsmMap` renders **OpenStreetMap raster tiles** directly onto a Compose `Canvas` — no Play Services, no API key, no map SDK. Tapping the map adds polygon vertices exactly as the Google path does, so zones can be drawn, validated and saved with no key at all.
- `MercatorProjection` is a pure, JVM-tested implementation of the standard slippy-map projection (world = `256 · 2^zoom` px, latitude through the Gudermannian). Rendering is restricted to integer zoom levels so the 256-px tiles line up exactly with the projected polygons and markers.
- Tiles come from **three independent, key-free providers** (`TilePolicy.PROVIDERS`), each with its own mirror hosts — **OpenStreetMap (the default)**, Esri World Street Map and OpenTopoMap. OpenStreetMap is the canonical key-free source and leads by default. CARTO Voyager was tried and **removed**: it now serves a "CARTO" watermark instead of a map under these request conditions, so it is not offered at all. A provider is a basemap *style*, so the map keeps **one provider for the whole viewport** (never a patchwork of styles) and only moves to the next when the current one fails. The last provider that worked is remembered and tried first, so **one blocked hostname no longer blanks the map**. A descriptive `User-Agent` and

each provider's required attribution are used; concurrent tile requests are capped to respect the OpenStreetMap usage policy.
- Fetched tiles are kept in a **size-bounded on-disk cache** (`TileDiskCache`, 40 MB cap, LRU by last use). Once an area has been viewed online it keeps rendering **with no network at all**, and the cache survives the app being closed. The eviction rule is pure and unit-tested.
- Tap the top-left provider caption (e.g. "OpenStreetMap ▸") for **map diagnostics**: cache size, current zoom/centre, a **"Check map servers"** button that probes **each of the three providers** and reports `ok (host)` / `HTTP …` per provider, and **"Clear map cache"**. Zoom in/out buttons sit in the top-right for anyone who finds pinch awkward.
- If tiles cannot be fetched at all the map degrades to a plain graticule grid plus a scale bar, so the screen is never blank and drawing always works.
- The camera opens **on the user**, never at `(0, 0)`. It prefers, in order: the live GPS fix → the newest draft vertex → a saved zone → the phone's **last-known location** (shared `rememberDeviceLocation()` helper). Only when all four are missing does it show a "Finding your location…" card. (It previously sat at null island, which is why the screen looked like a blank blue slab.) The map also **follows** the newest fix and draws a breadcrumb **trail**.
- The Google path now **follows** the live fix too (`FollowLatestFix`), with a my-location button on the Tracking screen to stop/start following.

Want the real Google basemap? See `docs/google_maps_setup.md` for the 5-minute key walk-through and troubleshooting.

Honesty note: tile *download* is a runtime, on-device behaviour. In this environment there is no device, so the fallback's projection maths is unit-tested on the JVM but tile rendering on a phone is **compile-checked only and has not been observed**. When a Google key *is* present, the Google Maps path is used instead.

---

7. Building and running (Windows PowerShell)

The toolchain used here lives on `D:`:

```
D:\AndroidTools\jdk-21        JDK 21 (Temurin)
D:\AndroidTools\gradle-8.11.1 Gradle
D:\AndroidTools\Sdk           Android SDK (platform 35, build-tools 35.0.0, platform-tools
D:\AndroidTools\gradle-home   GRADLE_USER_HOME
```

`build.ps1` sets `JAVA_HOME`, `ANDROID_HOME`, `ANDROID_SDK_ROOT`, `GRADLE_USER_HOME` and changes to its own directory, so it can be run from anywhere.

```
# Debug APK
powershell -NoProfile -ExecutionPolicy Bypass -File "D:\TouristSafetyMonitoring-src\build.

# Unit tests
powershell -NoProfile -ExecutionPolicy Bypass -File "D:\TouristSafetyMonitoring-src\build.

# Compile the instrumented-test sources without a device
powershell -NoProfile -ExecutionPolicy Bypass -File "D:\TouristSafetyMonitoring-src\build.

# Everything
powershell -NoProfile -ExecutionPolicy Bypass -File "D:\TouristSafetyMonitoring-src\build.
```

With a pass-through Maps key:

```
powershell -NoProfile -ExecutionPolicy Bypass -File "D:\TouristSafetyMonitoring-src\build.
```

7.1 The Gradle wrapper (optional, more portable)

A committed wrapper (`gradlew.bat` , Gradle 8.11.1) is included, so you can build with only a JDK on `PATH` — no separately installed Gradle. It still needs the SDK location and the same environment variables, which `build.ps1` sets for you:

```
$env:JAVA_HOME        = "D:\AndroidTools\jdk-21"
$env:ANDROID_HOME     = "D:\AndroidTools\Sdk"
$env:ANDROID_SDK_ROOT = "D:\AndroidTools\Sdk"
$env:GRADLE_USER_HOME = "D:\AndroidTools\gradle-home"
Set-Location "D:\TouristSafetyMonitoring-src"

.\gradlew.bat :app:assembleDebug
.\gradlew.bat :app:testDebugUnitTest
```

The first `gradlew.bat` run downloads the Gradle 8.11.1 distribution once into `GRADLE_USER_HOME\wrapper\dists` .

7.2 Running the authority backend and dashboard (optional)

The app works standalone; run the backend only if you want an authority to receive events.

```
python backend\run.py                    # dashboard: http://127.0.0.1:8080/  · health: /a
python backend\run.py --host 0.0.0.0     # reachable from a phone on the same network
```

```
python backend\run.py --port 9000 --database D:\tsm\authority.db
```

On first start it generates a tourist and an authority account and a **tourist sign-in PIN**, prints them **once**, and writes the passwords to `backend/var/DEV_CREDENTIALS.txt` (the PIN is not written there).

The app no longer asks a tourist for the address. A debug build can bake one in at build time (`-PAUTHORITY_URL=http://192.168.1.102:8080`, or `TSM_AUTHORITY_URL`, or `AUTHORITY_URL=` in `local.properties`); it is written to the session store on first launch and *never* overwrites an address the user has already chosen. The APK attached to this pass is built with `http://192.168.1.102:8080`, so it works out of the box on the same Wi-Fi as this machine. If the address needs changing, it lives in **Settings → Backend connection (advanced)** (emulator: `10.0.2.2:8080`; physical phone: the LAN address `run.py` prints), where **Test connection** performs a real `GET /api/v1/health` — the screen says *Connected* only when something actually answered.

Then, on the phone: choose **I'm a tourist → Create an account**, fill in a name, phone number, a PIN and confirm it (emergency contacts are optional), and the service allocates a Tourist ID such as `TR-2026-00124`, shown with a copy button. Sign in with that id and the PIN (the id and password **or** the id and PIN also work for an account an officer created). No officer action is needed for the registration to appear under **Registered Tourists** on the dashboard, or in the app's own **I'm an authority officer → register** view. Turn on **Settings → Authority reporting** if you want routine GPS position reports (a documented interval/distance policy) sent as well as crossings and SOS. Full detail — routes, configuration and the sync contract — is in `backend/README.md`.

Install on a connected device (USB debugging enabled):

```
& "D:\AndroidTools\Sdk\platform-tools\adb.exe" install -r "D:\TouristSafetyMonitoring-src\
& "D:\AndroidTools\Sdk\platform-tools\adb.exe" shell am start -n com.touristsafety.app/.Ma
```

The debug APK built by this pass is **20.33 MB** (21,317,914 bytes), SHA-256 `F0786379EC1917F9004E03F72E9317FB0C0F97B4B7D4BF9F312A911D3BE5D239` (copies were also placed in the project root, on the Desktop and in `Downloads` as `app-debug.apk`). It is a debug build, so it is signed with the debug key and is **not** suitable for distribution.

> **Installing over an older copy:** use `adb install -r …` (or install the APK over the existing app without uninstalling). The debug key is unchanged, so this keeps the stored sessions and the saved authority address. It is the **same** package, and the previous APKs could not reach the authority — see the note on plain HTTP below.

7.3 Plain-HTTP authorities and Android's cleartext policy

The authority is a plain-HTTP server on the local network. **Android 9 (API 28) and newer block cleartext HTTP by default** for any app whose `targetSdk` is 28+, and this app targets API 35. A build without a `networkSecurityConfig` therefore fails *every* call with

```
Cleartext HTTP traffic to 192.168.1.102 not permitted
```

…before a single byte reaches the network — which the sign-in screens can only report as "the authority is not answering". No amount of restarting the server fixes that: the phone is refusing to send the request.

`app/src/main/res/xml/network_security_config.xml` (referenced from the manifest as `android:networkSecurityConfig`) sets `cleartextTrafficPermitted="true"` precisely so a LAN authority is reachable. If the authority is ever moved behind TLS, flip it back to `false` and use an `https://` address. `NetworkSecurityConfigTest` fails the build if the manifest and the policy ever drift apart.

---

8. Testing

8.1 Automated tests

Plain-JVM (no emulator needed):

- `PolygonGeometryTest` — ray casting, concave edges, boundary distance, area, self-intersection.
- `PolygonContainmentServiceTest` — the accuracy-aware policy (too coarse, on the edge, straddling).
- `PolygonValidatorTest` — too few vertices, duplicates, zero area, bow-ties.
- `GeoFenceEngineTest` — entry/exit, no repeated alerts, hysteresis, overlapping zones, safe zones.
- `PolygonCodecTest` — ordered, lossless and fail-soft encoding.
- `SimulationPathBuilderTest` — the generated route is outside → inside → outside.
- `MercatorProjectionTest` — the key-less map projection: round-trips, known tile indices, ground resolution, panning.
- `TilePolicyTest` — tile URLs (incl. `{z}/{y}/{x}` order), three independent providers (OpenStreetMap, Esri, OpenTopoMap — CARTO is refused because it now watermarks), provider-scoped cache keys, LRU cache eviction, the OpenStreetMap concurrent-request cap, and the fail-over rule that drops a provider already known to be unreachable.
- `TileDiskCacheTest` — the on-disk tile cache against real files: byte-exact round-trip, size/count, LRU trim, clear.

- `SampleAreasTest` — the "add sample areas" generator: three valid areas, one per type, laid out near the user without overlapping.
- `OutboxPayloadsTest` — the JSON/offline payloads and their dedupe keys.
- `ApproachRiskTest` — the published approach-risk weights: severity ordering, capped history, proximity, night/dusk, neutral handling of an unavailable model, band boundaries at 30/60, and the invariant that the score never exceeds 100 **and** the listed factors account for every point of it.
- `ZoneHistoryTest` — per-zone history: entry/exit counts, 7- and 30-day windows measured against the supplied clock, future timestamps never counted, one-day-one-event collapsing.
- `ApproachBriefingTest` — which boundary gets briefed (higher risk wins; ties resolve to the closer zone), safe zones never briefed, disabled zones ignored, zero distance when inside, no fix → no briefing.
- `SosDeliveryStateTest` — the watch delivery indicator: a settled outcome is never walked back by a late "still queued" observation, while a newly raised SOS resets it.
- `NetworkSecurityConfigTest` — reads `AndroidManifest.xml` and `res/xml/network_security_config.xml` and fails if the app stops declaring a profile that permits cleartext, or declares one that forbids it: the manifest/policy pair that decides whether a plain-HTTP LAN authority can be reached at all.

Robolectric-backed (real Room/SQLite on the JVM):

- `TrackingPipelineTest` — the real GPS pipeline end to end against a scripted location source.
- `RoomPersistenceTest` — durability across close/reopen, CRUD, pruning, simulated-data isolation, the versioned 1→2→3 migration contract, and an executed run of **both** migrations on a real SQLite database proving a pre-upgrade row survives.
- `SimulationControllerTest` — the simulator drives the real engine and stamps every row.
- `GeoFenceEditorViewModelTest` — the drawing and saving rules, including typed-in coordinates and the explanation shown when a tap is refused.
- `OutboxQueueTest` — offline queue durability, dedupe on retry, and one-time delivery.
- `PositionReporterTest` / `PositionReportPolicyTest` — routine position reporting against a real SQLite outbox: a real fix becomes exactly one `POSITION` row, a **simulated** fix and an out-of-range coordinate are refused, nothing is queued until an address is configured (and then the pending fix is sent at once), and the interval + distance throttle reports a ~222 m move but suppresses an ~11 m nudge.
- `WatchControllerTest` — SAFE→WARNING haptics once per entry, SOS creation, honest delivery status.
- `ForestReferenceTest` — **the shipped model files themselves**: the activity and impact forests in `assets/` are re-walked on the JVM against real recorded windows and must reproduce scikit-learn's own probability vectors and argmax, and both must index the same 24 features in the same order as the Python extractor.

- `MovementAnalyzerTest` — the three fall gates (impact ≥ 2 g, model score ≥ 0.55, stillness after the impact), that a still phone and an unrecognised impact are **never** falls, that one fall seen by five overlapping windows raises exactly one alert, and that an impact landing at the end of a window is not failed for a tail it cannot measure.
- `MovementControllerTest` — one sensor subscription only, stop is repeatable, a device without both sensors is reported rather than pretended away, and a build without the trained model says so.

Future-event risk model (see §9.6):

- `FutureRiskFeaturesTest` — the 22 features index by index on signals whose values can be worked out by hand (a resting phone reads exactly 1 g and nothing else; a second of free fall is 20/50 below 0.5 g; a quarter turn reads 90°), the "no net direction" guard that returns 0 instead of dividing by ~zero, and the refusals for a short window or mismatched channels.
- `FutureRiskReferenceTest` — **the shipped asset itself**, re-walked on the JVM against scikit-learn's own probabilities for **16 real held-out windows**: feature parity, forest parity (worst 5.9e-04 on a step function, mean < 5e-04), the Platt calibration, and the refusal to load an artefact whose feature **names** do not match the extractor's.
- `FutureRiskAnalyzerTest` — band mapping at each threshold, the **calibrated** probability (not the raw vote), insufficient data, an implausible (> 16 g) or non-finite reading, staleness, a paused watch, factor ordering, and that the reported factors carry a non-causal caveat.
- `FutureRiskReportPolicyTest` — first elevated, escalation (never throttled), the 120 s refresh, exactly one "cleared" record, and that `UNAVAILABLE`/stale bands are never shared; robust to a clock jump.
- `FutureRiskReporterTest` — against a real SQLite outbox: one durable `RISK` row, a **summary** rather than a sensor stream, nothing queued when no address is set, offline queue + recovery, and dedupe.
- `FutureRiskPayloadTest` — the wire contract through a real JSON parser: null probability, non-finite handling, escaping, and that **no coordinates or raw samples** ever travel in it.
- `FutureRiskMovementIntegrationTest` — the movement watch feeds both models from **one** subscription, a missing model reports "prediction unavailable", a HIGH forecast raises **no** fall alert, and stop keeps the last forecast.
- `FutureRiskCardModelTest` — what the card may say: no band when there is no prediction, **never** "low" for "we do not know", thresholds printed with their provenance, the non-causal caveat, and that it never claims to summon help.
- `AuthorityLiveBackendTest` (5th case) — opt-in, against a **real server**: it queues a risk summary built by the production payload builder, delivers it, reads it back from `/api/v1/authority/risk` as a `PREDICTION`, and asserts the SOS total does not move. Run it with `-Dtsm.e2e.baseUrl=…`; it is one of the five that skip in an ordinary run.

Authority client and backend:

- `AuthorityApiTest` — the wire codec: base-URL normalisation, the login body, a **PIN** login body (PIN carried, password omitted), the set-PIN body, the sync body (payload as a nested object; a corrupt payload kept as text so the server can reject it with a reason), every delivery verdict, the officer's **register** rows (registration / tracking / safety kept as three separate claims, a position only when one really exists, a row with no Tourist ID dropped rather than rendered anonymously), and the server's own error code/message surfaced verbatim.
- `AuthoritySyncTransportTest` — the retry/reject rules: `DELIVERED` and `DUPLICATE` are success, `REJECTED` and HTTP 400 are terminal with a reason, and 401/403/500/dead-network/no-address/no-sign-in all **keep the event queued**. Also the selector: the honest no-backend transport until an address exists.
- `AuthorityHttpServerTest` — the client over a **real TCP socket** to a hand-rolled HTTP/1.1 server, proving method/path/bearer-header/body, the officer register fetched with the officer token, a tourist token refused the same route (403), a closed port is `Unreachable`, a gateway error page is an `HttpError`, and a truncated body is a protocol error.
- `AuthoritySessionStoreTest` — the real on-disk session store (no Robolectric, a throw-away file): the address and both sessions come back after the store is torn down and rebuilt, a build-time default **never** overwrites a chosen address, and a tourist sign-out cannot evict an officer (or the reverse).
- `OfficerViewModelTest` — the officer role end to end through the ViewModel: sign-in stores an officer session *without* signing a tourist in; a rejected sign-in stores nothing and says only that the pair was refused (so officer ids cannot be enumerated); an unreachable authority is named; blank credentials and a blank address are refused before any call; a 403 on the register drops the session; and sign-out revokes the token while leaving the tourist session alone.
- `JsonTest` — the hand-rolled codec: nested objects/arrays/unicode round-trip, whole numbers written without a decimal point (so the backend's integer validator accepts them), truncated/trailing documents rejected.
- `EmergencyContactRepositoryTest` — the contact list: priority from list position, whole-list replace with no partial write, null relationship, quote escaping, delete.
- `AuthorityLiveBackendTest` (opt-in) — seven tests against a real server you start: SOS delivery + idempotency + the officer queue, a **position report** landing on the authority map, **PIN set + PIN sign-in** (a wrong PIN is 401), **self-registration → the server-minted Tourist ID → signing in with it → appearing under *Registered Tourists* on the officer's API with no position yet → gaining one when a real report arrives**, **the in-app officer console reading the register through the app's own client method while a tourist token is refused it with 403**, and a dead backend leaving the event queued with its reason. Skipped unless `-Dtsm.e2e.baseUrl` is supplied.

Backend (Python, `python -m pytest backend/tests -q`): **265 tests** across `test_registration.py` (45), `test_payloads.py` (42), `test_authority.py` (33), `test_sync.py` (29), `test_risk.py` (28), `test_auth.py` (26), `test_pin.py` (18), `test_app.py` (16), `test_contacts.py` (15) and `test_security.py` (13), plus `python backend/verify_integration.py` → **41/41** against a real

subprocess server (the batch still carries an on-device `RISK` summary, proven to reach `/authority/risk` and to raise no SOS, and the run now also self-registers a tourist, signs in with the minted id and PIN, checks the officer's register shows it as `SELF` / `REGISTERED` with no position, refuses a duplicate phone (409) and a role-escalating registration (400), and confirms the account survives a restart).

Run it with `:app:testDebugUnitTest` (see §7). The HTML report lands in `app/build/reports/tests/testDebugUnitTest/index.html`.

The suite was executed in this environment with `:app:testDebugUnitTest` and reported **548 tests, 0 failures, 0 errors, 7 skipped** (50 classes). The seven skips are the opt-in live-backend tests, which run only when a server address is passed with `-Dtsm.e2e.baseUrl`; run that way against the real server started for this pass, **all seven passed** (see `AuthorityLiveBackendTest` above):

Test class	Tests
`AuthorityApiTest`	28
`PolygonGeometryTest`	22
`TilePolicyTest`	21
`FutureRiskAnalyzerTest`	18
`FutureRiskCardModelTest`	18
`PolygonValidatorTest`	18
`RegistrationFormTest`	18
`GeoFenceEngineTest`	17
`AuthoritySyncTransportTest`	15
`GpsStatusTest`	15
`AuthorityHttpServerTest`	14
`GeoFenceEditorViewModelTest`	14
`FutureRiskFeaturesTest`	13
`PolygonContainmentServiceTest`	13
`TrackingPipelineTest`	13
`FutureRiskReporterTest`	12
`RoomPersistenceTest`	12

Test class	Tests
`TrackingRecoveryTest`	12
`AuthorityViewModelTest`	11
`GeofenceDiagnosticsTest`	11
`MercatorProjectionTest`	11
`PlaceholderWatchTest`	11
`SimulationPathBuilderTest`	11
`ApproachBriefingTest`	10
`ApproachRiskTest`	10
`FutureRiskReportPolicyTest`	10
`MovementAnalyzerTest`	10
`OfficerViewModelTest`	10
`OutboxPayloadsTest`	10
`GeofenceWalkthroughTest`	9
`JsonTest`	9
`PolygonCodecTest`	9
`AuthoritySessionStoreTest`	8
`FutureRiskMovementIntegrationTest`	8
`FutureRiskPayloadTest`	8
`OutboxQueueTest`	8
`SimulationControllerTest`	8
`AuthorityLiveBackendTest`	7 (7 skipped by default)
`EmergencyContactRepositoryTest`	7
`MovementControllerTest`	7
`NetworkSecurityConfigTest`	3
`FutureRiskReferenceTest`	6
`PositionReporterTest`	6

Test class	Tests
`PositionReportPolicyTest`	6
`SampleAreasTest`	6
`TileDiskCacheTest`	6
`WatchControllerTest`	6
`ZoneHistoryTest`	6
`SosDeliveryStateTest`	4
`ForestReferenceTest`	3
Total	**548**

There is also a Python suite. From the repo root: `python -m pytest ml/tests ml/future_risk/tests -q` → **179 passed** — 86 GeoLife/UCI-HAR/OSM/weather/ trajectory-history/transport tests, plus **93** for the future-risk pipeline (feature arithmetic, label geometry rebuilt from the raw recordings, leakage, calibration, the size budget, app-asset byte-equality and the model-selection rule). And the authority backend suite: `python -m pytest backend/tests -q` → **265 passed**.

8.2 What is *not* automated

There is no emulator or AVD in this environment, so the instrumented (`androidTest`) sources are **compile-checked only** and have **not been executed**, and map rendering has not been exercised at runtime. That includes:

- `AppDatabaseInstrumentedTest` — on-device persistence smoke test.
- `DatabaseMigrationTest` — opens a v1 database, runs `MIGRATION_1_2` , and asserts a saved zone survives and the `outbox_events` table is created (also validates the hand-written SQL against the exported schema).
- `StatusBannerInstrumentedTest` — Compose UI test for the status banner.

Also not executed: **Google Maps rendering with a real key**, the new **Create/Tracking centring** and **follow toggle**, the **"Add 3 sample areas" button**, the **on-disk tile cache serving a map offline after a restart**, the **"Check map servers" diagnostic / provider fail-over** and the **zoom buttons** — all need a device. The geometry, the cache's file behaviour and the provider policy *are* unit-tested (`SampleAreasTest` , `TileDiskCacheTest` , `TilePolicyTest`).

Added in the 2026-10-17 bug-fix round and likewise **device-only**: the **blue "you are here" marker** (its colour is a theme constant, but whether it is *visible* over the tiles can only be judged on a phone), the **closer default zoom**, the **latitude/longitude entry row** as rendered, the **crash-report dialog**, and the **tile-memory bounding / request de-bouncing** that is meant to stop the app being killed

while panning. The coordinate *parsing and refusal messages* and the *provider fail-over rule* are unit-tested; the rest is compile-checked only.

Added later on 2026-10-17 and also **device-only**:

- The `CameraUpdateFactory is not initialized` **crash fix** — the reported "blue marker appears, then the app closes". The guard is compile- and JVM-verified, but only a device can confirm the *reported* crash is gone (recorded as **INST-16 NOT EXECUTED**).
- **Live sensor inference** — accelerometer + gyroscope → the 24 features → the shipped forest → the three fall gates. The maths is JVM-tested against recorded windows; no sensor has ever been read by this project (**INST-17**).
- The **movement/fall card** and the **approach-briefing card** as rendered, and the **distance-to-boundary pill** on the map (**INST-18**, **INST-19**). Their state is unit-tested (MOV-01…MOV-18, RISK-01…RISK-12); the pixels have not been seen.
- The **default key-free basemap** as rendered — the three providers answered `HTTP 200` from the build machine, but the rendered basemap is not observable here.
- The **Welcome / role-chooser**, **Tourist sign-in + Create account** and **Authority officer console** screens as rendered, and the device's own path to the backend over emulator loopback / Wi-Fi (**INST-20**, **INST-21**, **INST-24**). The logic behind them is JVM- and real-socket-tested, and the live test drives the same client methods against a real server here; the pixels and the device's HTTP stack have not been exercised.
- The **Authority** and **Emergency-contacts** screens as rendered.

Added on 2026-10-18 and also **device-only**:

- The **future-risk card** as rendered — the horizon, the band, the calibrated probability, the contributing factors and the two honest states ("Insufficient data", "Prediction unavailable"). Its *wording* is pinned by `FutureRiskCardModelTest` (18 tests) and its *inference* by `FutureRiskReferenceTest` (6), both on the JVM; the pixels have never been seen (**INST-22**).
- **Live sensor → future-risk inference** — accelerometer → the 50-sample window → the 22 features → the shipped forest → the calibrated band. `FutureRiskMovementIntegrationTest` covers the wiring against recorded windows and the shipped asset is proven against scikit-learn, but no sensor has ever been read by this project (**INST-23**).
- The **authority dashboard's "Future-risk predictions" panel** as rendered. `app.js` is syntax-checked with `node -e`, and the rows it renders are asserted at the API level by `backend/tests/test_risk.py` (28 tests), but the panel has never been opened in a browser here.

Everything in §8.1 runs on the JVM with Robolectric.

8.3 Real-device GPS test plan

1. Install the debug APK on a physical phone and open the app.
2. Settings → grant **location** and **notifications**; confirm all four indicators turn green.
3. **Multiple areas:** Manage Geo-Fences → **Add 3 sample areas near me**. Three labelled polygons (restricted / dangerous / safe) should appear near you; zoom out on Create/Tracking to see all three. Delete them afterwards.
4. Create a zone around your current position (tap three or four corners, *Close ring*, name it, pick *Restricted*, *Save*). The map should open **on your area**, not on the ocean.
5. Tracking → **Start tracking**. Confirm the fix appears, the **trail** grows, the camera **follows**, and the my-location button stops/starts following. The status banner turns red as you cross into the polygon and back to green when you leave.
6. Walk out and back in. Exactly **one** alert per crossing should appear — no repeats while you remain inside.
7. Check the persistent notification appears while tracking, and that pressing *Stop tracking* on it ends tracking.
8. Switch on **background tracking**, grant "*Allow all the time*" in system settings, lock the screen, and confirm alerts keep arriving.
9. Toggle **airplane mode** on and confirm tracking and alerts keep working (only map tiles stop).
10. Add a second overlapping zone and confirm each raises its own alert.
11. Settings → *Delete simulated data only* and confirm the real history is untouched.
12. **No-key map:** with `local.properties` containing *no* `MAPS_API_KEY`, open Create Geo-Fence and confirm OpenStreetMap tiles load, tapping adds vertices, and a zone can be saved and reloaded.
13. **Map diagnostics & offline cache:** tap the provider caption and press *Check map servers* — each of the three providers should read `ok`. Then turn on **airplane mode**, close and reopen the app and return to the same area: previously viewed tiles should still render from the on-disk cache.
14. **Google map:** add a key (see `docs/google_maps_setup.md`), rebuild, and confirm the Create/Tracking maps show Google tiles centred on you.
15. **Smartwatch:** open the watch from the top app bar, walk into a restricted zone, and confirm the phone buzzes once (short double buzz) and the face turns **WARNING**. Press **SIMULATED SOS** and confirm the long SOS pattern. With no authority address configured the delivery status must read **"Queued (offline)"**, never "Delivered"; with an address configured and a tourist signed in it must show the server's real verdict (see steps 21–23).
16. **Offline queue:** turn on airplane mode, cross a boundary, then restore the network; confirm the queued count appears and events are retried. With no backend configured they remain queued and the app says so; with one configured they drain to the authority on the next flush.
17. **Blue "you are here" marker:** start tracking and confirm a **blue** dot with a white collar is visible over the tiles (it used to be drawn in the theme's dark green and was almost impossible to see against a *safe* zone). With tracking off it must instead be a **hollow grey** ring, which means "last known only" — never a solid dot.

18. **Map opens close in:** open Create Geo-Fence without drawing anything. The view should be at walking scale (about a kilometre across, zoom 17) so a drawn boundary covers the place you meant, not a town. Use the **+ / −** buttons to adjust, and tap the map again to confirm that one tap adds exactly one corner, immediately, with a "Vertex n placed" read-out.
19. **Corner by coordinates:** in Create Geo-Fence, press **Fill from my position** and confirm the latitude/longitude boxes fill in with the sign already correct; edit a digit, press **Add point**, and confirm the corner appears and the vertex count rises. Typing nonsense must produce a readable reason, not silence.
20. **If the app ever closes by itself:** reopen it. A **"Last run ended unexpectedly"** dialog should show the recorded stack trace with a **Copy** button — send that text on. With no crash, no dialog appears.
21. **Panning must not kill the app:** drag and pinch the map continuously for ~30 seconds at several zoom levels. Previously this grew the in-memory tile map without limit; it should stay responsive and the app must not be killed.
22. **The reported crash must be gone:** with no `MAPS_API_KEY` set, open Tracking and start tracking. The moment the **blue marker** appears is exactly where the app used to close. It must stay open. If it ever does close, reopen it and copy the stack trace from the "*Last run ended unexpectedly*" dialog — that text is the whole point of the dialog (INST-16).
23. **Movement & fall watch:** on Tracking press **Start** in *Movement & fall watch*, grant nothing (the accelerometer and gyroscope need no permission), and walk for ~20 seconds. The card should show an activity with a confidence percentage, the window length, and a steadily rising *Windows analysed* count. Lying the phone flat and still should report *Laying* / *Standing* rather than a walk.
24. **The fall gates must not fire on a bump:** while walking, tap the phone against your palm. Peak impact should rise but, unless the impact is ≥ 2 g **and** the model score is ≥ 0.55 **and** the phone then goes still, **no** suspected fall should appear. A single high score with no impact must never raise anything. Confirm the card always says **"suspected"** and that **no SOS is sent**.
25. **Approach briefing:** walk to within ~200 m of a restricted or dangerous zone. The **approach briefing** card should appear with the zone name, the distance to its boundary, *your own* recorded crossings of that zone, and **every** risk factor with its points. The **pill over the map** must show the same distance and score. Standing inside should switch it to "Inside …".
26. **Basemap style:** with no Google key, confirm the tracking/Create maps render a street basemap and that the provider caption reads **"OpenStreetMap ▸"**. Tap the caption → *Check map servers*: all three providers should report ok.
27. **Authority backend:** start it with `python backend\run.py`, open http://127.0.0.1:8080/ and confirm the dashboard loads. On the phone, open **Settings** → **Backend connection (advanced)**, enter `10.0.2.2:8080` (emulator) or the printed LAN address (physical phone), press **Test connection** and confirm it says **Connected** (and that an address with no server behind it says **Offline** instead). Sign a tourist in with the id and password from

`backend/var/DEV_CREDENTIALS.txt`, switch the toggle to **PIN**, press **Set or change PIN**, and sign in again with the id + PIN — a wrong PIN must be refused.

28. **SOS delivery:** press **SIMULATED SOS** in the watch and confirm the phone reports it delivered, the SOS appears in the dashboard's queue badged as **simulated**, and it can be acknowledged there.
29. **Emergency contacts:** add a contact from **Home** → **Emergency contacts**, then confirm it appears in the dashboard for that tourist (it uploads through the queue, so it also works offline and syncs later).
30. **SOS acknowledgement reaches the tourist:** acknowledge the SOS in the dashboard, then reopen **Settings** → **Backend connection (advanced)** on the phone and confirm the latest-SOS row shows the acknowledged state, and the *position on file* row shows the coordinates the authority holds.
31. **Routine position reporting:** in **Settings** → **Authority reporting**, switch it on and pick an interval, then walk a short distance with tracking running. Confirm the dashboard's tourist detail shows a moving latest position, and that with no address configured nothing is queued.
32. **Home status cards:** on Home, confirm the **GPS status** card distinguishes the states (waiting for permission / acquiring / live / poor accuracy) with the fix age and accuracy, and that the **your account** card shows the Tourist ID, the name, and how many alerts are waiting to send.
33. **The role screen (first launch):** wipe the app's data (or install fresh) and open it. The first screen must ask **who is holding the phone** with two choices, not drop straight into a dashboard.
34. **The button is never silently dead:** choose **I'm a tourist** → **Create an account** and press **Create my account** with the form empty and with the PINs deliberately mismatched. Each field must carry its own message ("Enter your name.", "The two PINs do not match.") and a line under the button must name exactly what is still missing — the button must never be greyed out with no explanation, and it must not depend on a backend address having been typed anywhere.
35. **A real account, a real id:** complete the form (name, phone, a 4-digit PIN twice; contacts optional), press **Create my account**, and confirm the success screen shows a **Tourist ID** in the form `TR-2026-00124` with a **Copy Tourist ID** button. Press it twice — the second attempt must be refused as a duplicate phone, not silently accepted.
36. **Sign in with the minted id:** leave the success screen, sign in with that Tourist ID and the PIN you chose. A wrong PIN must be refused with the "not accepted" wording; the correct one must land on the safety home showing that Tourist ID under **your account**.
37. **The session survives a restart, and logout works:** force-stop the app and reopen it — it must go straight to the safety home, not the role screen. Then **Home** → **Sign out**: the app must return to the role screen, and after that *Continue* must no longer be offered.
38. **The officer console, and the wall around it:** from the role screen choose **I'm an authority officer**, sign in with the officer id and password from `backend/var/DEV_CREDENTIALS.txt`, and confirm the register lists the tourist you just created (registration date, **Awaiting first report**,

no location). Then sign out, sign in as the *tourist*, and confirm the officer console is not reachable from the tourist path at all.

---

9. Machine learning: what is real, and what it is not

Four things in this app are sometimes called "the AI", and they are **not** the same kind of thing. The distinction is stated here, in the app, and in the docs, because a safety app that oversells its own certainty is worse than one that says nothing.

Thing	Kind	Honest description
Trajectory anomaly score (GeoLife)	**Unsupervised model**	A real `IsolationForest` over real GPS data. Output is "*this movement is unusual*", **never** danger and **never** a probability. GeoLife has no anomaly labels, so no accuracy/F1 against real anomalies exists or is claimed.
Activity + impact detection (UCI HAR)	**Supervised models**	Two real random forests with **real labels** for activity, and **synthesised** impacts for the impact class. Shipped as JSON and walked on-device in Kotlin — no TensorFlow, no TFLite, no network.
Future risk prediction (UniMiB SHAR)	**Supervised future-event model**	A real random forest predicting a **hard fall impact within the next second**, from **real recorded falls** (30 subjects). Runs **on-device** with the same JSON walker, shows a **calibrated** probability and a LOW/MODERATE/HIGH band, and is **not a deployment-ready fall alarm** (§9.6).
Approach-risk score	**Not a model**	A published, unit-tested weighted checklist. It has no labelled outcome data to learn from and is not dressed up as a probability.

These are three *different* things the screens keep apart: the **geofence rule**, the **approach-risk checklist**, and the **trained future-event prediction**. Only the last is a model that predicts an event before it happens.

9.1 Activity recognition (real labels)

24 time-domain features computed on the phone (`MotionFeatures.kt`), mirroring the Python extractor exactly and in the same order. Random forest, 32 trees, depth 8, under 263 KB.

- **Accuracy 0.8351 / macro-F1 0.8333** on **2947 held-out windows from 9 subjects never seen in training** (the dataset's own subject-disjoint split, verified `subject_overlap: []`).
- Reference 561-feature forest: 0.9287. Single decision tree: 0.7967. Both are reported, because either alone is misleading.

- The three *walking* variants are the confusion; the app only asks the coarser "is this person moving?" question, which is exactly the distinction that is reliable.

9.2 Impact / fall-signature detection (**synthesised positives**)

> **UCI HAR contains no falls.** No fall class, no impact annotation, no subject who fell. The positive class was **manufactured**: a fall-like transient (free-fall dip, impact spike, then stillness) injected into real recorded windows from held-out subjects; the negative class is the same recordings untouched.

- ROC-AUC **0.9999**, average precision **0.9999**, F1@0.5 **0.9962**.
- **Shuffled-label control ROC-AUC 0.5198** — chance. This is what shows the near-perfect score is the detector finding the injected signature, not leakage.
- **It is still not a fall-detection claim.** An injected impact is a large stereotyped transient; a real fall can be gentle, or look like sitting down quickly. That is exactly why §9.6 uses **real recorded falls** (UniMiB SHAR) for the future-risk model instead of extending this one.
- The model's score is **one of three gates** and never sufficient alone: **impact ≥ 2.0 g**, **model score ≥ 0.55**, and **stillness after the impact (std ≤ 0.12 g)** — measured from the *last* sample at or above the impact threshold. A 15-second cooldown stops one fall raising five alerts. A high score with no physical impact raises **nothing**, and **no SOS is ever sent automatically**.

9.3 Approach-risk briefing (not a model)

Shown within **200 m** of a restricted or dangerous boundary: the zone, the distance, **your own** recorded crossings of that zone, and a 0–100 score with **every factor itemised**. Published weights: zone severity (DANGEROUS 40 / RESTRICTED 28 / SAFE 5), 3 per crossing in the last 7 days capped at 18 (4 for never-visited ground), proximity up to 16, night 12 / dusk 5, ambulating 4, and the live impact score 12 above threshold (6 neutral when no model is available). Bands: LOW < 30 ≤ MODERATE < 60 ≤ HIGH. The card says to your face that this is a checklist, not a learned probability.

9.4 Verification, and how to reproduce it

```
cd D:\TouristSafetyMonitoring-src
python ml\har_preprocess.py     # extract/verify UCI HAR -> ml\outputs\har\clean\
python ml\har_train.py          # activity model, metrics, figures -> ml\outputs\har\train
python ml\har_fall.py           # injected-impact detector      -> ml\outputs\har\fall\
python ml\har_verify_export.py  # exported JSON vs scikit-learn: expect ALL PASS
python ml\osm_eda.py            # real OpenStreetMap EDA         -> ml\outputs\osm\      (
python ml\weather_eda.py        # real weather EDA               -> ml\outputs\weather\  (
python ml\har_eda.py            # UCI HAR figure set             -> ml\outputs\har\eda\
python ml\history.py            # trajectory + zone-crossing trends -> ml\outputs\history\
```

```
python ml\transport_mode.py     # supervised transport-mode model -> ml\outputs\transport\
python ml\future_risk\train.py  # supervised FUTURE fall-impact model -> ml\outputs\future
python ml\future_risk\tune.py   # the 65-config validation search   -> ml\outputs\future_r
python -m pytest ml\tests ml\future_risk\tests -q   # 179 tests: artefact contracts + the
```

- `har_verify_export.py`: max probability error **4.97e-07** (activity) / **4.74e-07** (impact), **0** argmax mismatches — the JSON walker agrees with scikit-learn.
- `ForestReferenceTest` independently checks the **shipped** assets on the JVM against baked-in scikit-learn probabilities from real windows.
- Training is seeded, so re-running reproduces the metrics and regenerates `har_activity_model.json` **byte-identical** to the shipped copy.

Full method, per-class tables, embedded confusion matrices and limitations: `docs/model_card_movement.md`.

9.5 Datasets, and the honest answer on "future risk"

All four datasets named in the brief are analysed with **real, reproducible scripts** whose artefacts quote only measured numbers — full detail in `docs/dataset_and_eda.md`:

Dataset	What was done	Real numbers (measured)
GeoLife GPS trajectories	cleaning, 12-item EDA, unsupervised anomaly model, historical/temporal/zone-crossing analysis	18,670 trajectories / 182 users; 24,876,978 rows scanned, 0 malformed; distance 1,021,454 km; derived zone with **431** crossing trajectories
UCI HAR	figure set, supervised activity + impact models	561 features; 7,352/2,947 windows; subject-disjoint 21/9; accuracy **0.8351**, macro-F1 0.8333
OpenStreetMap	live EDA via the Overpass API (a key-free bounded extract, cached and attributed)	1.5 km² Tiananmen box: **12,783 nodes / 3,197 ways**, 5,681 nodes/km²
Weather	two sites over 2011-01-01 … 2012-07-27	Open-Meteo (key-free) — the brief's OpenWeatherMap needs a key that was not provided, and every artefact **says which provider it used**

Two things are stated rather than faked:

- **None of the four datasets contains an injury, danger or incident outcome**, so no supervised tourist-incident model can be trained from them, and inventing labels would be

fabrication. That is why §9.6's future-risk model does **not** use them: it uses a fifth dataset with **real outcome labels** (UniMiB SHAR's recorded falls), and says so.
- `ml/transport_mode.py` **is a transport-mode classifier, not a risk model** — it uses the only valid supervised target in the data (GeoLife's mode labels), scoring **0.8475 accuracy / 0.5784 macro-F1** on held-out *users*.

9.6 Future risk prediction — a real supervised model over real falls

> **The question:** given the last second of phone accelerometer data, will a hard fall impact land within the **next** second?

This is the one model in the app that forecasts an event rather than reacting to it. It is trained, not configured; it is separate from the geofence rule and from the approach checklist; and it is honest about being far from a shippable alarm.

Dataset. UniMiB SHAR (Micucci, Mobilio, Napoletano, 2017) — 30 subjects, 9 activities of daily living and **8 real fall types**, 50 Hz phone accelerometer. 1,980 trials read, **1,979 used**: one recording (subject 10, `Jumping`, trial 1) has 9,369/9,624 non-finite values and was **dropped, counted and reported** rather than repaired. No OSI licence exists for this dataset; the provenance file says so.

Target and horizon. Positive = a window ending 1–50 samples **before** a sample whose magnitude is the trial's `argmax` and is ≥ **2.0 g** (the app's own `FallPolicy.MIN_IMPACT_G`). 6 fall trials never reach 2 g and are excluded and counted. Horizon and feature window are both **50 samples = 1.0 s**, which is the **longest fully supported** horizon (trials are 151–446 samples with the impact around index ~120). A 30-minute horizon is **not attempted** — accelerometer data cannot see that far.

No leakage. A window ending at `t` uses `[t-49, t]`; the label asks about `(t, t+50]` — disjoint by construction, re-derived from the raw recordings by `train.check_no_future_information()` on every run. A **shuffled-label control** scores **ROC-AUC 0.477** (chance), which is what shows the pipeline leaks nothing.

Measured on held-out subjects (subject-disjoint 60/20/20, seed 42, 18/6/6 subjects; 51,128 windows, 6,458 positive):

Model	Precision	Recall	F1	ROC-AUC	PR-AUC
Majority class	0.0000	0.0000	0.0000	0.5000	0.128565
Logistic regression	0.4760	0.8432	0.6085	0.904389	0.659633
Hist gradient boosting	**0.7916**	0.6721	**0.7270**	**0.945913**	**0.815000**
Random forest, calibrated (shipped)	**0.6707**	**0.7498**	**0.7080**	**0.936848**	**0.764569**

- **Trial sensitivity 0.929577** — 264 of 284 real held-out falls were flagged at least once inside the last second. Window recall (0.7498) is reported *separately* because overlapping windows flatter it.
- **False alarms: 5.09 per minute** of ordinary activity at the app's cadence (16.30/min at the raw window stride), concentrated in *jumping* (0.161), *sitting down* (0.156) and *lying down* (0.127) — the three activities where a body really does drop under gravity.
- Calibrated with **Platt scaling fitted on the validation split only** (Brier **0.062600** → **0.056675**); bands **HIGH ≥ 0.314046** (validation max-F1) and **MODERATE ≥ 0.069201** (largest validation threshold keeping recall ≥ 0.95). `UNAVAILABLE` is a separate state and is never mapped to LOW.
- The quality winner (val PR-AUC **0.875909**) was **rejected for size**: it exports to 20,059.7 KB. The shipped model (val PR-AUC **0.845112**) exports to **1,968,314 bytes** — only **1 of 65** searched configurations fits the app's 2,000,000-byte asset budget, and the **−0.031 PR-AUC cost is in the card** along with the fact that `hist_gradient_boosting` (0.8150) beats what ships and cannot run on-device.

On the phone. Inference runs **on-device**, reusing the app's existing `DecisionForest` JSON walker (the only change is an optional `expectedFeatures` parameter) — **no new ML framework, runtime or format**. It shares the movement watch's **existing** accelerometer subscription, using the last 50 samples of each window, so the tourist-facing prediction works **fully offline** and no sensor stream leaves the phone. A throttled **summary** (never raw samples) goes to the authority via the existing outbox as `OutboxKind.RISK`, is stored in its own `risk_assessments` table and shown in its own dashboard panel — never mixed into the SOS queue. **A HIGH band never triggers an SOS.**

The honest bottom line: real model, real held-out numbers, real leakage controls — and **not a deployment-ready fall alarm**. Full model card, provenance, reliability table, per-activity false-alarm breakdown and all 11 limitations: **`docs/future_risk_prediction.md`**.

---

10. Known limitations

- No emulator was available here, so map rendering and instrumented tests are unverified at runtime.
- The key-less OpenStreetMap fallback's **projection maths, provider policy and on-disk cache are unit-tested**, but on-device **tile rendering has not been observed** (no device here).
- The tile cache is capped at 40 MB of the app's *cache* directory; Android may clear it under storage pressure, in which case tiles are simply re-fetched next time there is network.
- Third-party tile providers (OpenStreetMap, Esri, OpenTopoMap) are used only as fallbacks under their public terms, with attribution shown on the map. If you rely on this app, host your own tiles or configure a Google Maps key (`docs/google_maps_setup.md`).
- Map tiles require the network; containment, storage and alerts do not.

- The simulated smartwatch's **vibration can only be felt on a physical phone** — an emulator has no vibrator.
- **The authority backend ships here, but the device's own path to it is unproven.** `backend/` is a working Flask + SQLite service with an operator dashboard, and the production Kotlin client was driven against it on the JVM (41/41 acceptance checks, 6/6 live tests, including self-registration → the minted Tourist ID → signing in with it → appearing on the officer's register). What has **not** been done is the same flow *from a device* — its HTTP stack, emulator loopback / Wi-Fi routing, and the Compose sign-in and registration screens (INST-20, INST-21, INST-24). With no address configured, queued SOS/alert events stay `PENDING` ("Queued (offline)") and are retried; the app never pretends a message was delivered when it wasn't.
- **No device here means the live GPS path is unverified.** Routine `POSITION` reporting exists behind a documented interval/distance policy, and the reporter refuses simulated fixes and out-of-range coordinates in unit tests, but the real GPS that would drive it was never run on hardware.
- **The live OpenStreetMap and weather fetches need the network.** They succeeded in this round and the artefacts are cached; if every mirror fails the scripts write `NOT_RUN.md` rather than inventing data. The OpenWeatherMap branch is implemented but was **not exercised** (no API key).
- OEM battery managers can throttle a backgrounded app (see §4).
- **No real falls were used to train the impact detector** (§9.2). Its positives are synthesised impacts injected into real recordings, so its metrics measure that signature, not clinical fall detection. This is the single biggest limitation in the project. (The **future-risk model in §9.6 does use real recorded falls**, and it says so; but see the bullet below for what *its* numbers still do not mean.)
- **The future-risk model is not a deployment-ready fall alarm** (§9.6). Measured on held-out subjects it raises **5.09 false alarms per minute** of ordinary activity (16.30/min at the raw window stride), concentrated in *jumping* (0.161), *sitting down* (0.156) and *lying down* (0.127); its mid-range probabilities (0.2–0.4) are over-confident; it was trained on 30 healthy adults in a lab, never on a tourist outdoors; it forecasts **one second** ahead and nothing longer; and it **cannot distinguish a dropped phone from a falling person** (no phone-drop activity exists in the dataset, so that failure mode is untested). It is **not the best model in the repository** — `hist_gradient_boosting` reaches PR-AUC 0.8150 but cannot be walked on-device. A HIGH band is an **informational UI state** and a queued summary: it raises **no** alarm, changes **no** operational state, and **never** sends an SOS.
- **UniMiB SHAR carries no explicit OSI licence.** It is distributed by its authors for academic research, and the provenance file records that position rather than inventing one. Only derived artefacts are redistributed here, never the recordings.
- **UCI HAR is lab data** — 30 volunteers, scripted activities, waist-mounted phone — and the activity model is 0.8351 accurate, meaning roughly one window in six is misclassified. The UI therefore always shows the activity *with its confidence* and never acts on activity alone.

- **The approach-risk score is a weighted checklist, not a fitted model**, and must not be read as a probability (§9.3).
- **Nothing sensor-driven has been observed on hardware by this project** (no device available), so live inference, the new cards (including the future-risk card), the map pill and the crash fix are compile/JVM-verified only (INST-16…INST-23).
- This is a monitoring aid, not an emergency service: it does not contact anyone automatically.